

Descrição do Sistema

O **TechFix SGC** é um sistema de gestão comercial desenvolvido para controle de clientes, produtos e vendas. Ele oferece uma interface web para operações administrativas e funcionalidades via API REST, com autenticação JWT e perfis de acesso diferenciados. O sistema gerencia o cadastro de clientes e produtos, controla vendas com atualização automática de estoque e gera relatórios gerenciais.

Requisitos Funcionais

Código	Descrição
RF01	Cadastrar, listar, buscar, atualizar e excluir clientes
RF02	Cadastrar, listar, buscar, atualizar e excluir produtos
RF03	Registrar, listar e consultar vendas
RF04	Atualizar estoque automaticamente ao realizar uma venda
RF05	Gerar relatórios de vendas por período, cliente e faturamento anual
RF06	Autenticar usuários com login e senha (JWT)
RF07	Controlar acesso com perfis (Administrador e Funcionário)
RF08	Impedir exclusão de clientes que possuem vendas
RF09	Calcular automaticamente o valor total da venda
RF10	Validar CPF único e preço de produto não negativo

Requisitos Não Funcionais

Código	Descrição
RNF01	Desenvolvido em Python com Django 4.2
RNF02	Banco de dados SQLite (escolhido por simplicidade)
RNF03	Arquitetura baseada em Django REST Framework para API

RNF04	Interface web com HTML, CSS e JavaScript
RNF05	Autenticação segura com JWT
RNF06	Sistema deve ser executado em ambiente local
RNF07	Código organizado em apps modulares (clientes, produtos, vendas, relatorios)

Arquitetura Proposta

O sistema segue uma arquitetura **cliente-servidor** baseada em Django, organizada em:

1. **Camada de Apresentação:** Templates HTML + CSS + JavaScript
2. **Camada de Negócio:** Apps Django (clientes, produtos, vendas, relatorios)
3. **Camada de API:** Django REST Framework com endpoints RESTful
4. **Camada de Autenticação:** JWT via API /auth/login
5. **Camada de Persistência:** SQLite (ORM Django)

Navegador (Frontend) ↔ Django (Backend/API) ↔ SQLite (Banco de Dados)

Padrões de Projeto Escolhidos

- **MVC (Model-View-Controller)** - Através do padrão MTV (Model-Template-View) do Django.
- **REST (Representational State Transfer)** - Para construção da API.
- **Repository Pattern** - Implementado indiretamente pelo ORM do Django.
- **Dependency Injection** - Fornecido pelo sistema de apps e configurações do Django

Diagrama de Classe (Simplificado)

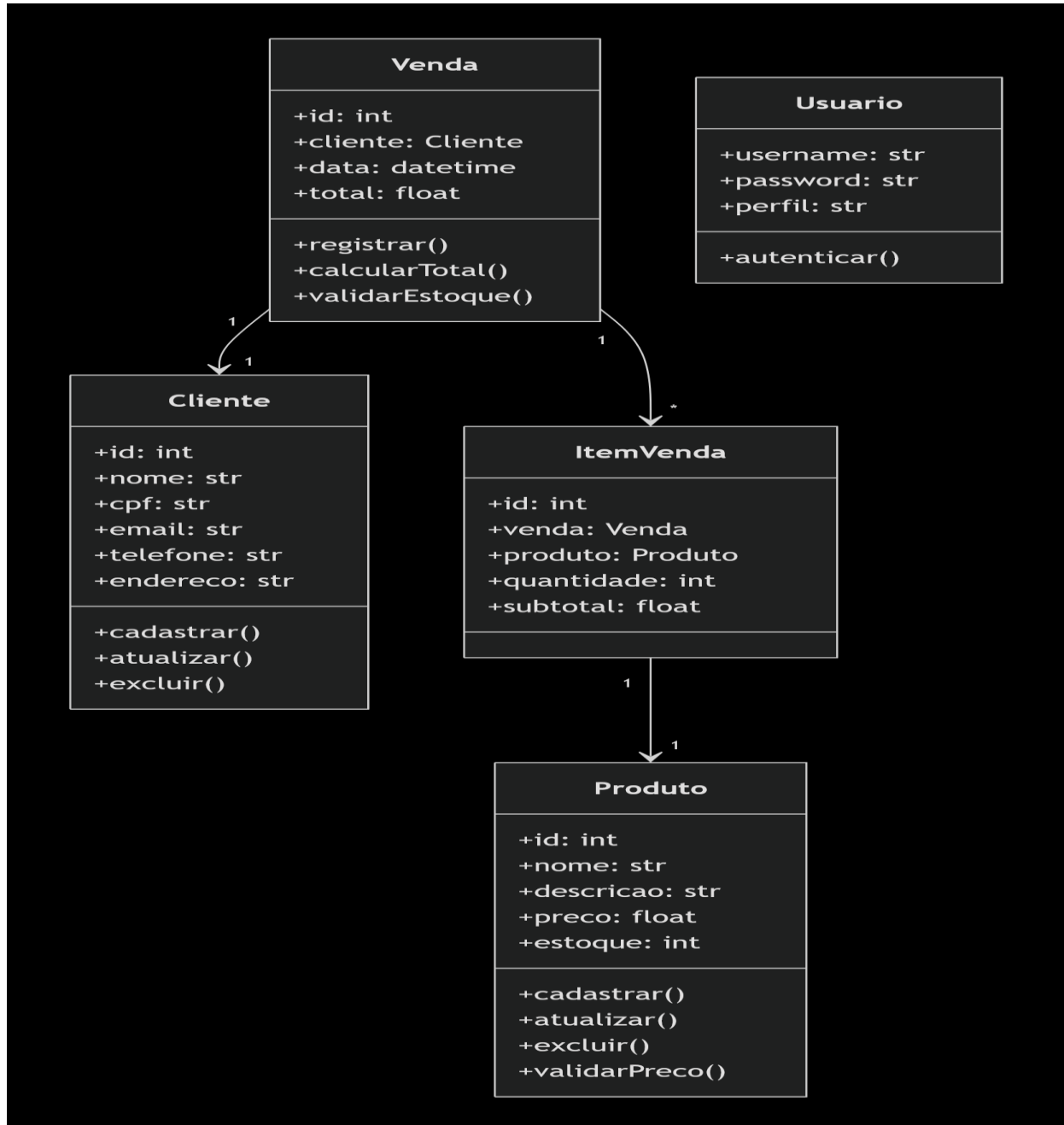
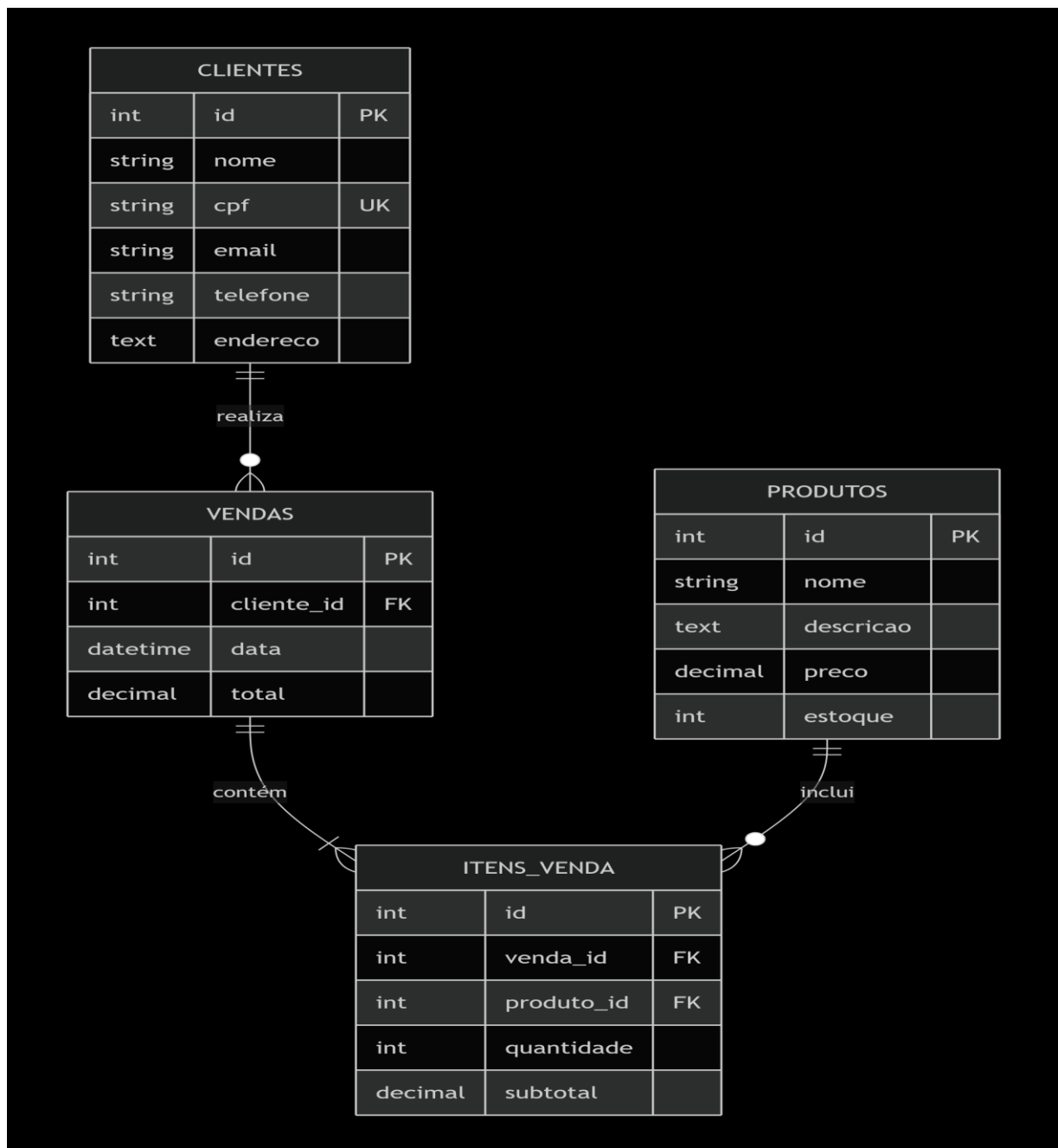


Diagrama de Domínio

O domínio do sistema é composto por quatro entidades principais:

- **Cliente:** Pessoa física que realiza compras. Possui atributos como nome, CPF, e-mail, telefone e endereço.
- **Produto:** Item comercializado. Possui nome, descrição, preço e quantidade em estoque.
- **Venda:** Representa uma transação comercial. Está associada a um cliente, possui data, um valor total e uma lista de itens.
- **ItemVenda:** Detalha os produtos em uma venda. Possui quantidade e subtotal para cada produto associado.

Diagrama Lógico do Banco de Dados



Cadastro de Clientes e Produtos (Exemplos via API)

Cadastro de Cliente (POST /api/clientes/)

json

```
{
  "nome": "João Silva",
  "cpf": "12345678901",
  "email": "joao@email.com",
  "telefone": "11999999999",
  "endereco": "Rua Exemplo, 123"
}
```

Cadastro de Produto (POST /api/produtos/)

json

```
{
  "nome": "Placa-Mãe TechFix",
  "descricao": "Placa-mãe modelo Z590",
  "preco": 799.90,
  "estoque": 15
}
```

Script de Criação do Banco de Dados (SQLite/Django ORM)

O banco de dados é gerado automaticamente pelo Django através das migrações. Os principais modelos (tabelas) são representados abaixo.

Modelo Cliente (clientes/models.py)

python

```
from django.db import models
```

```
class Cliente(models.Model):
    nome = models.CharField(max_length=100)
    cpf = models.CharField(max_length=11, unique=True)
```

```
email = models.EmailField()
telefone = models.CharField(max_length=15)
endereco = models.TextField()
def __str__(self):
    return self.nome
```

Modelo Produto (produtos/models.py)

python

```
from django.db import models

class Produto(models.Model):
    nome = models.CharField(max_length=100)
    descricao = models.TextField()
    preco = models.DecimalField(max_digits=10, decimal_places=2)
    estoque = models.IntegerField()

    def __str__(self):
        return self.nome
```

Modelo Venda e ItemVenda (vendas/models.py)

python

```
from django.db import models
from clientes.models import Cliente
from produtos.models import Produto

class Venda(models.Model):
    cliente = models.ForeignKey(Cliente, on_delete=models.PROTECT)
    data = models.DateTimeField(auto_now_add=True)
    total = models.DecimalField(max_digits=10, decimal_places=2,
default=0)

class ItemVenda(models.Model):
    venda = models.ForeignKey(Venda, on_delete=models.CASCADE,
related_name='itens')
    produto = models.ForeignKey(Produto, on_delete=models.PROTECT)
    quantidade = models.IntegerField()
    subtotal = models.DecimalField(max_digits=10, decimal_places=2)
```

Repositório GitHub

O projeto está disponível no repositório público:

https://github.com/LucasFloresss/TECHFIX_WEB2

Alunos: Lucas Gomes, Mariana Borel, Susana Borges, Gabriel Guedes e Marcos Vinicius